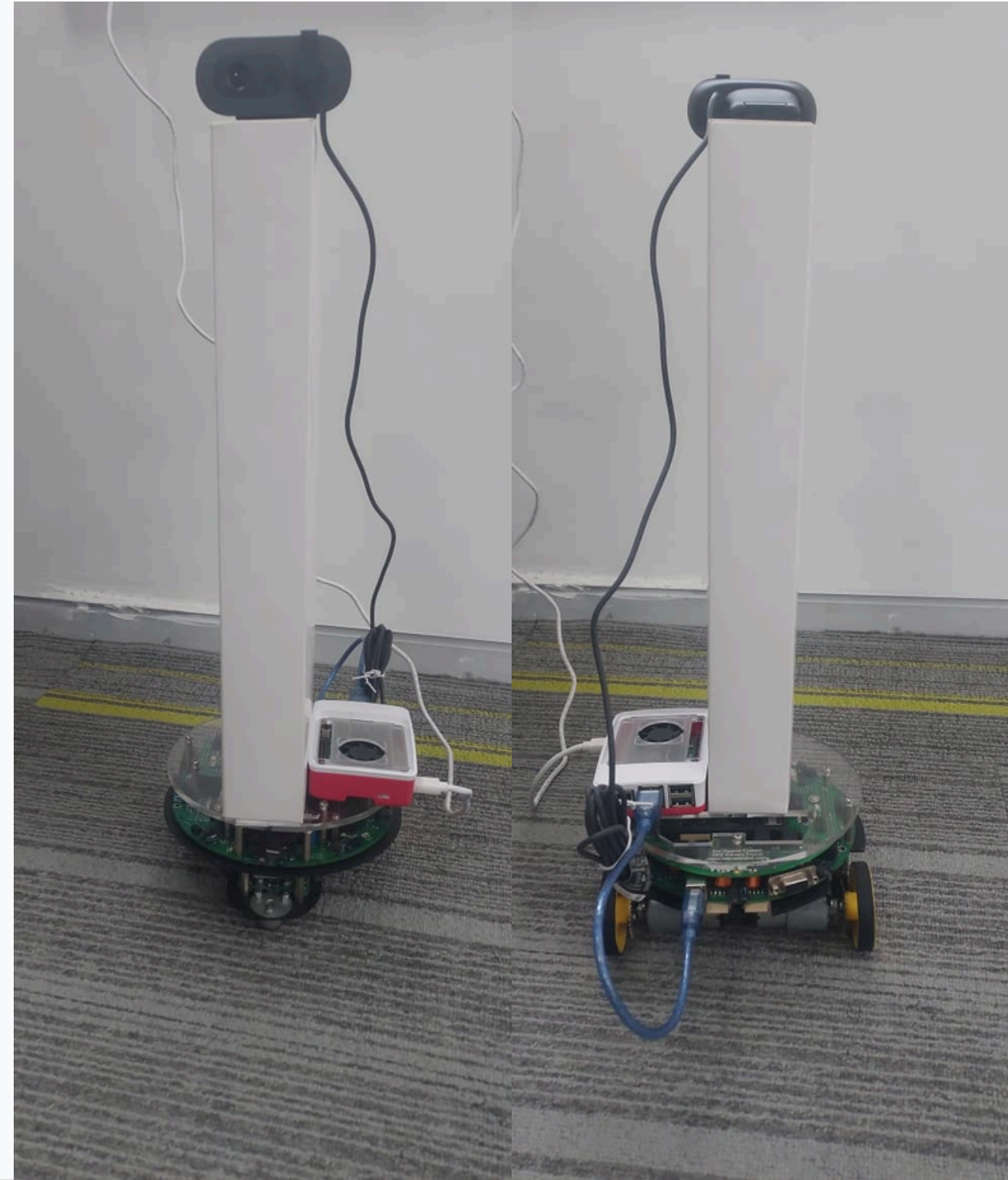


KPIT Apex Lab Internship | May – July 2026

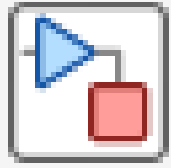
Face Detection & Tracking Robot

*Firebird V (ATmega2560) + Raspberry Pi 5
MATLAB / Simulink → Python + YOLOv8n*



Darpan Dongre • Pratham Modi • Amay Bembde

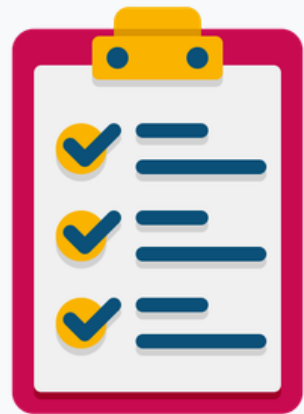
What we'll cover



Simulink

Phase 1 — Simulink build

Hardware, pipeline, tracking, watchdog



What we tried, what failed

8 approaches, root cause, Phase 1 results



Phase 2 — Python build

Threading, voice control, obstacle avoidance, demo

Part 1

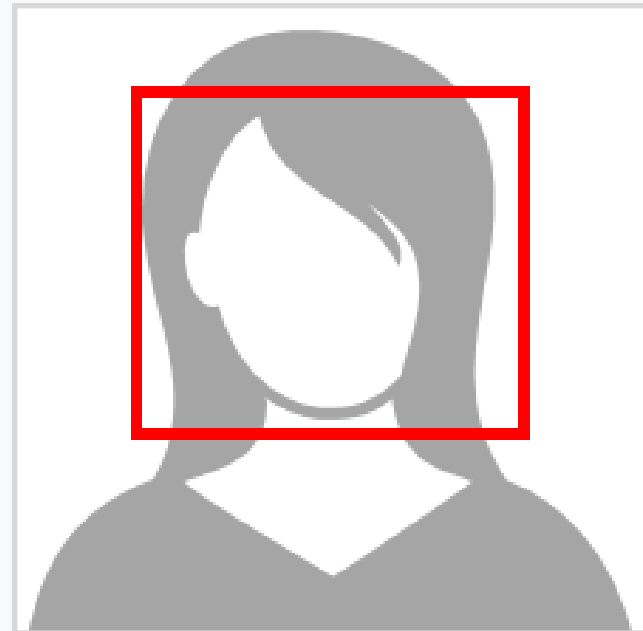
System Overview & Phase 1

The goal

Build a robot that sees a person, decides which way to turn, and drives — on its own, in real time, with no cloud or GPU.

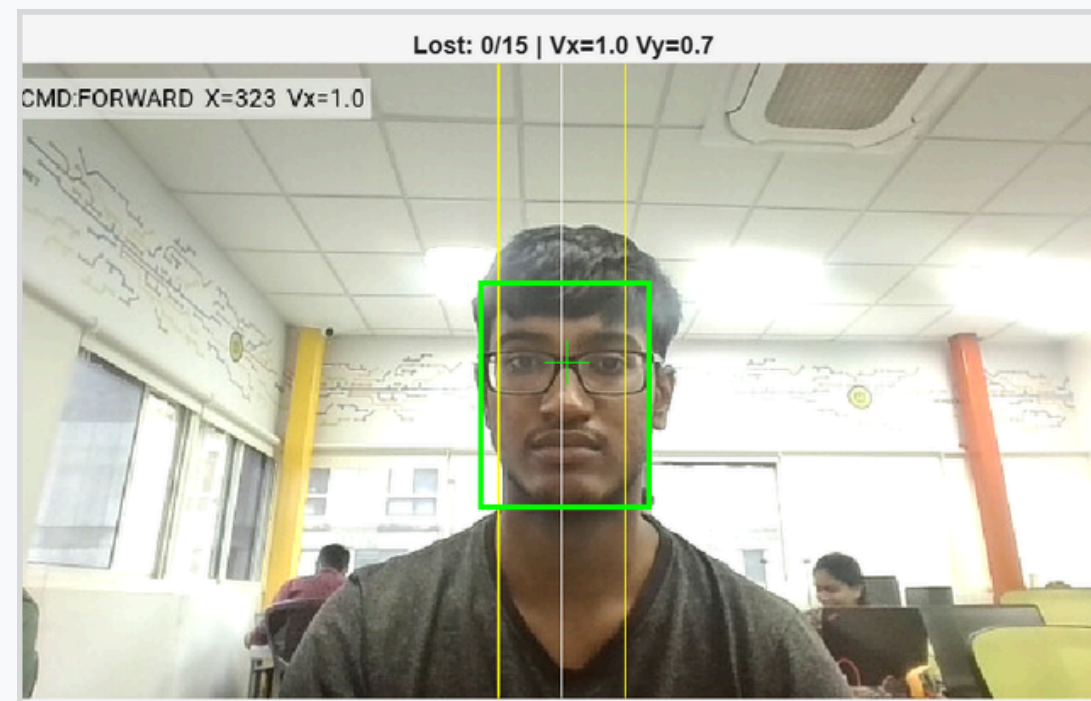
Perceive

Camera captures live video
YOLOv8n detects the person



Decide

Is the target left, right, or centred?
How far away is it?



Act

Send motor command over USB-UART
Robot steers to keep the target in frame

```
function charOut = cmdToSerial(cmdCode)
charOut = uint8(83); % default S

if cmdCode == int8(0)
    charOut = uint8(83); % S
elseif cmdCode == int8(1)
    charOut = uint8(70); % F
elseif cmdCode == int8(2)
    charOut = uint8(76); % L
elseif cmdCode == int8(3)
    charOut = uint8(82); % R
end
end
```

The hardware

Firebird V

ATmega2560 wheeled robot from eYantra. H-bridge motor driver. Listens for single-byte serial commands.

Raspberry Pi 5

ARM Cortex-A76, 4-core 64-bit. Runs all camera capture and inference. Talks to the robot over USB-UART.

USB webcam

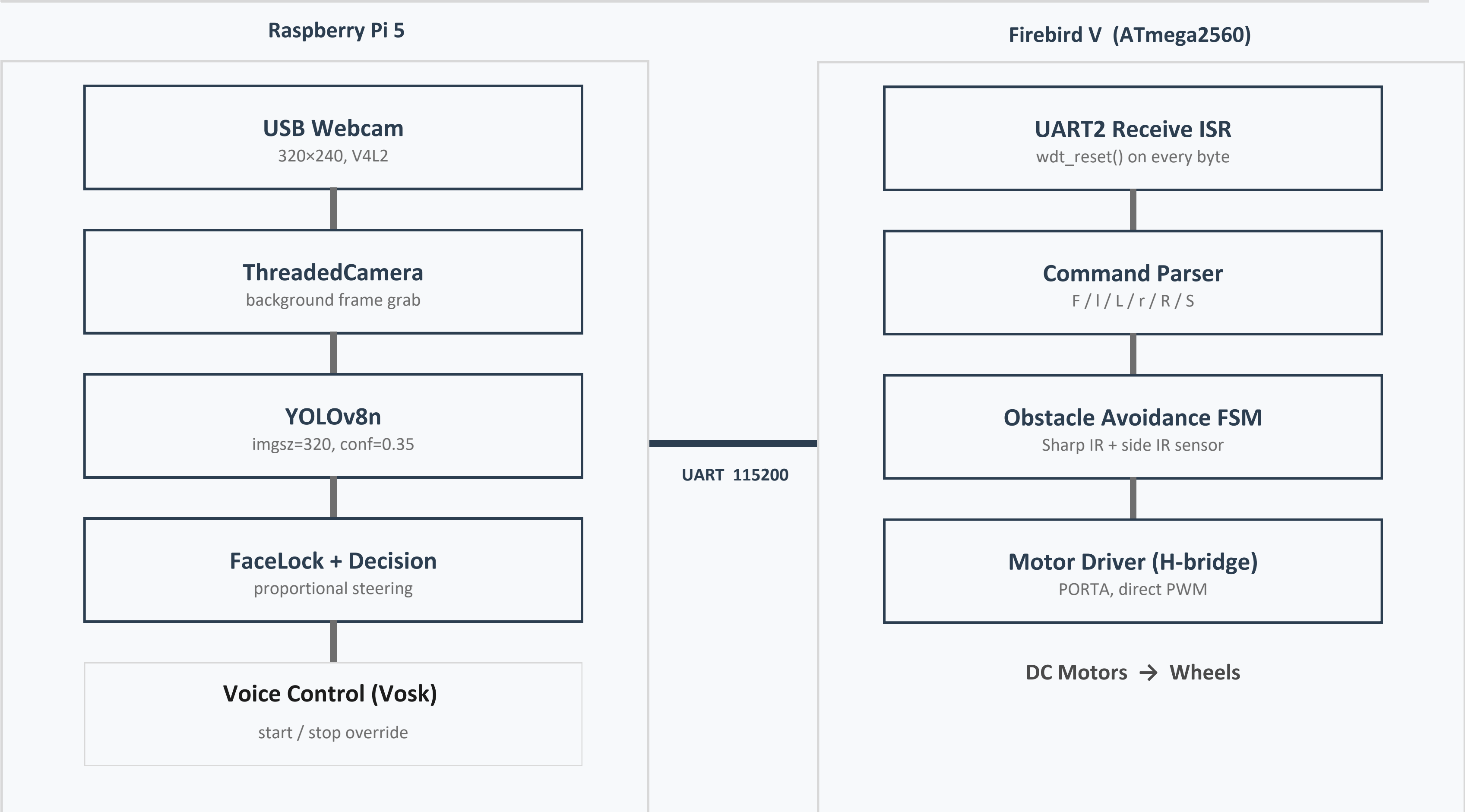
160×120 (MATLAB) → 320×240 (Python). V4L2 capture.

Host PC

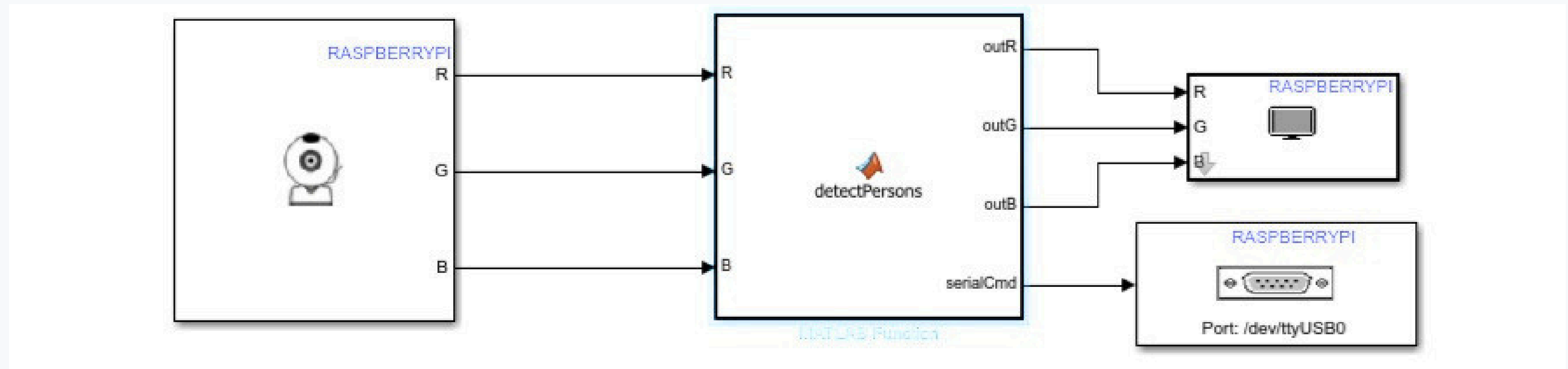
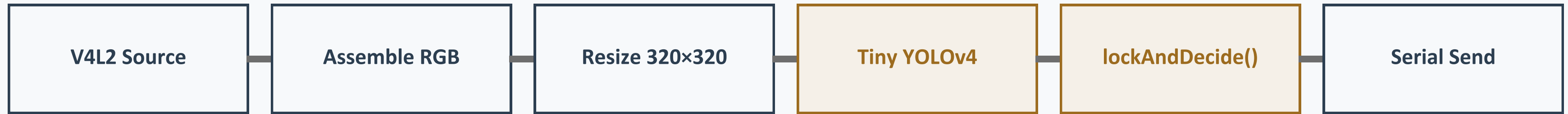
MATLAB phase only — build, codegen, deploy to Pi over Wi-Fi.

Pi 5 → UART 115200 → Firebird V

System architecture



Phase 1: the Simulink pipeline



Frame-skip

YOLO takes 700–800ms per frame. We run detection every 3rd frame and reuse the last box in between. A heartbeat byte goes out every frame to keep the watchdog alive.

IoU target lock

Lock onto the person nearest frame-centre. Hold lock if IoU > 0.25. Drop after 6 missed frames (~5 s) and stop.

'F' Forward · 'L' Left · 'R' Right · 'S' Stop — one byte at 115200 baud

Fail-safe: the hardware watchdog

If the Pi crashes, the robot must stop — not keep driving. The ATmega2560 runs a 1-second hardware watchdog. Every serial byte resets it. Silence for 1 second → MCU resets → motors stop.

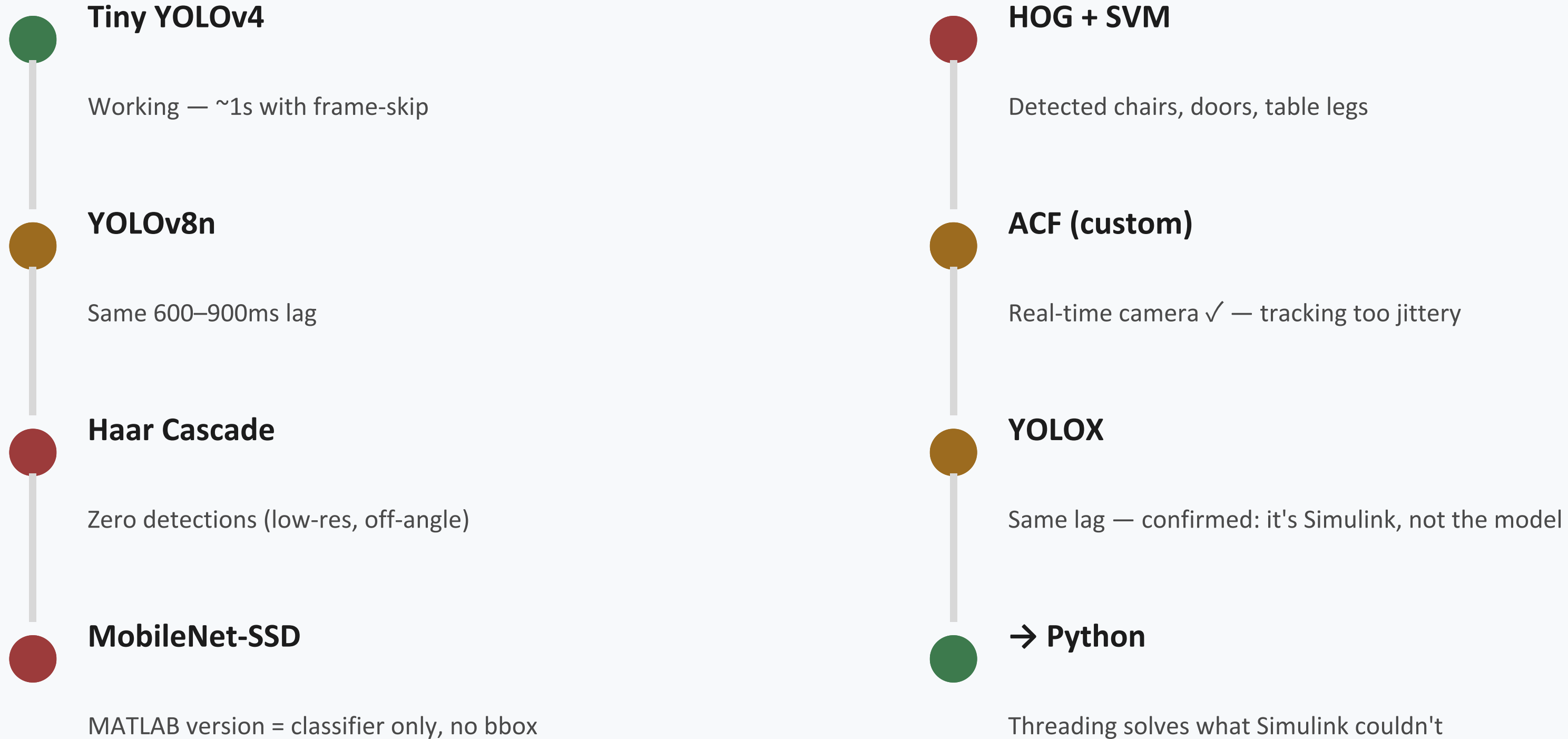
Why 1s and not 500ms? YOLO inference on the ARM CPU spikes to 700–800ms. A shorter timeout would trigger false resets during normal detection.

Single byte	Transport	Rate	Frequency
F / I / L / r / R / S	UART2, USB	115200 baud	every ~100–300ms

Part 2

What We Tried — and What Went Wrong

Eight approaches — one worked in MATLAB



So what was actually wrong?

It wasn't the model. Simulink's synchronous execution blocks the whole pipeline until detect() returns — regardless of which detector you use.

- The model step blocks until detect() returns
- 600–900ms per inference on the Pi's ARM cores
- YOLOv4, YOLOv8n, YOLOX — near-identical latency
- No async or threading in Simulink's codegen workflow
- Classical detectors were fast but too unreliable in our lab



Fix: decouple inference from the control loop using Python threading.

Phase 1 results

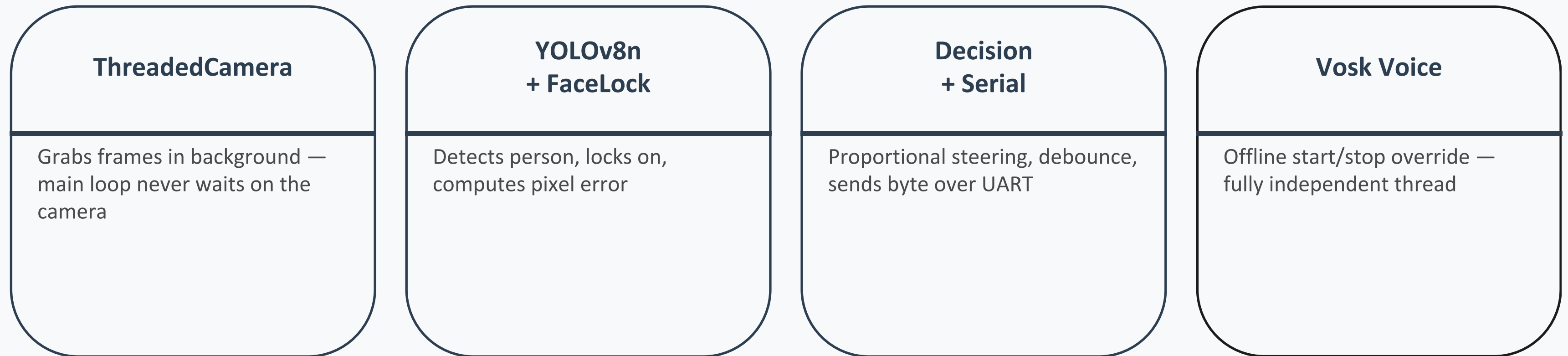
Test	Observed	Result
Camera lag — 32-bit OS	~10s lag	Fail
Camera lag — 64-bit OS	~8s lag	Partial
Camera lag — no display block	~7s lag	Partial
Camera lag — frame-skip ×3	~1s lag	Pass
Person detection accuracy	Box drawn correctly	Pass
Left / right turn trigger	Correct direction	Pass
Forward motion	Stays centred	Pass
Stop when too close	Robot stops	Pass
Watchdog (Pi killed mid-run)	Stops within 1s	Pass
No person in frame	Robot stops	Pass

Part 3

Phase 2: The Finished Python Build

Tracking · Voice control · Obstacle avoidance

Phase 2: four threads, one Python app



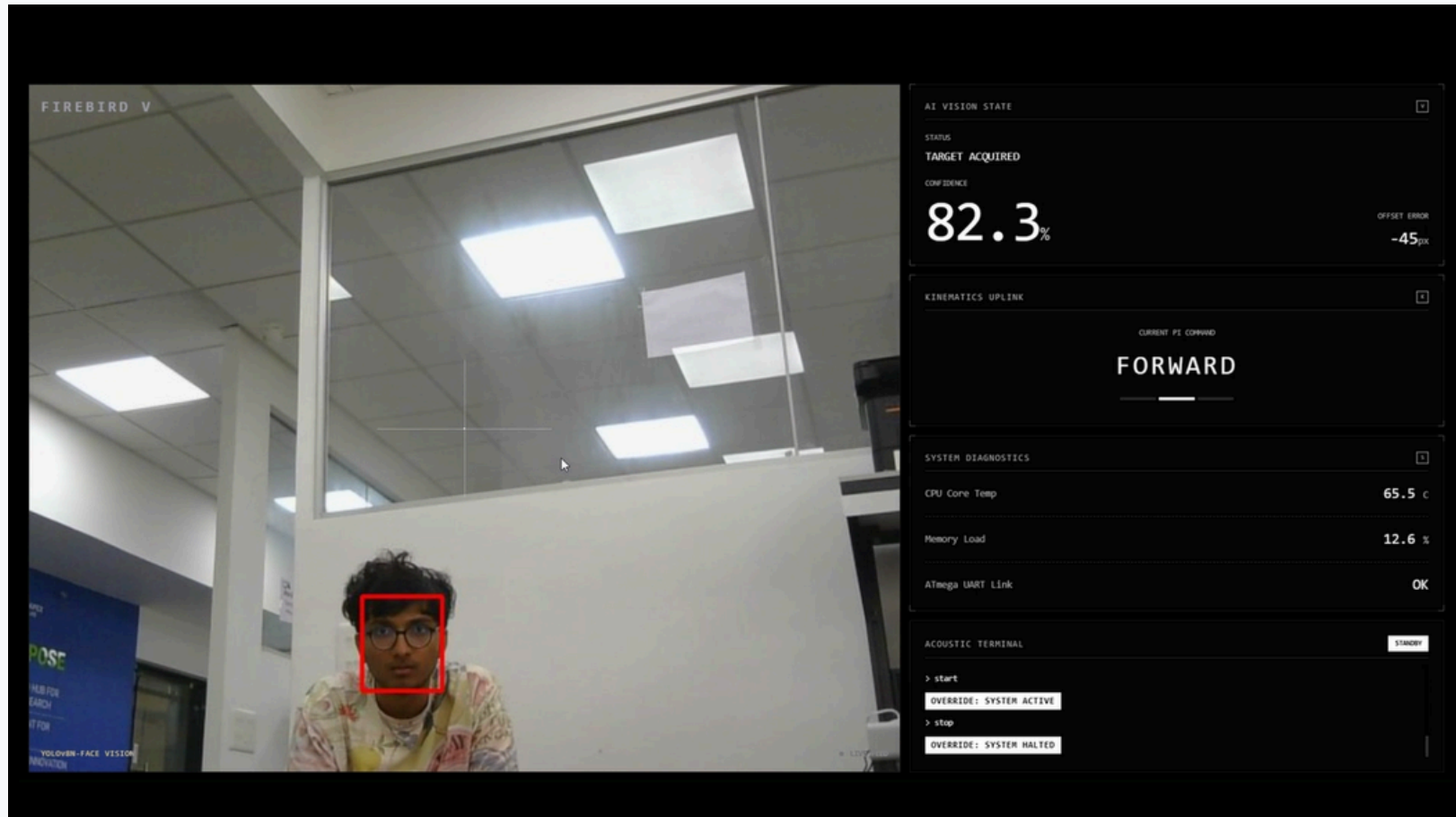
What this fixes vs Phase 1

Camera lag gone — ThreadedCamera always has a fresh frame ready · Inference no longer blocks serial sends · Voice can halt the robot from across the room · Obstacle avoidance runs as a firmware reflex on the ATmega underneath all of this

Locking on and steering

FaceLock

- Lock on nearest person to frame centre
- Hold by nearest centre-point distance (cheaper than IoU)
- 5-second grace period before dropping the lock
- Brief loss → repeat last command for up to ~2s
- Debounce: same decision 3 frames in a row, 80ms apart



Proportional steering

≤ 75px **F** straight ahead

75–115px **l / r** soft turn, reduced speed

> 115px **L / R** hard turn, full speed

Phase 1 was binary Left/Right. Three bands cut the wobble at the edge of the deadband.

Voice control — offline, two words

Once the robot is untethered there's no easy way to stop it from across the room. We added a voice layer: say 'start' or 'stop', no internet required.

Engine	Vosk — fully offline speech recognition, 16kHz via sounddevice
--------	--

Vocabulary	Intentionally just 'start', 'stop', and an 'unknown' bucket — fewer words = faster, more reliable on the Pi CPU
------------	---

Responsiveness	Polls partial results while you're still speaking — 'STOP' takes effect almost instantly
----------------	--

Override	Voice STOP is a hard override. While halted the main loop sends 'S' regardless of what the tracker wants
----------	--

Obstacle avoidance — a reflex

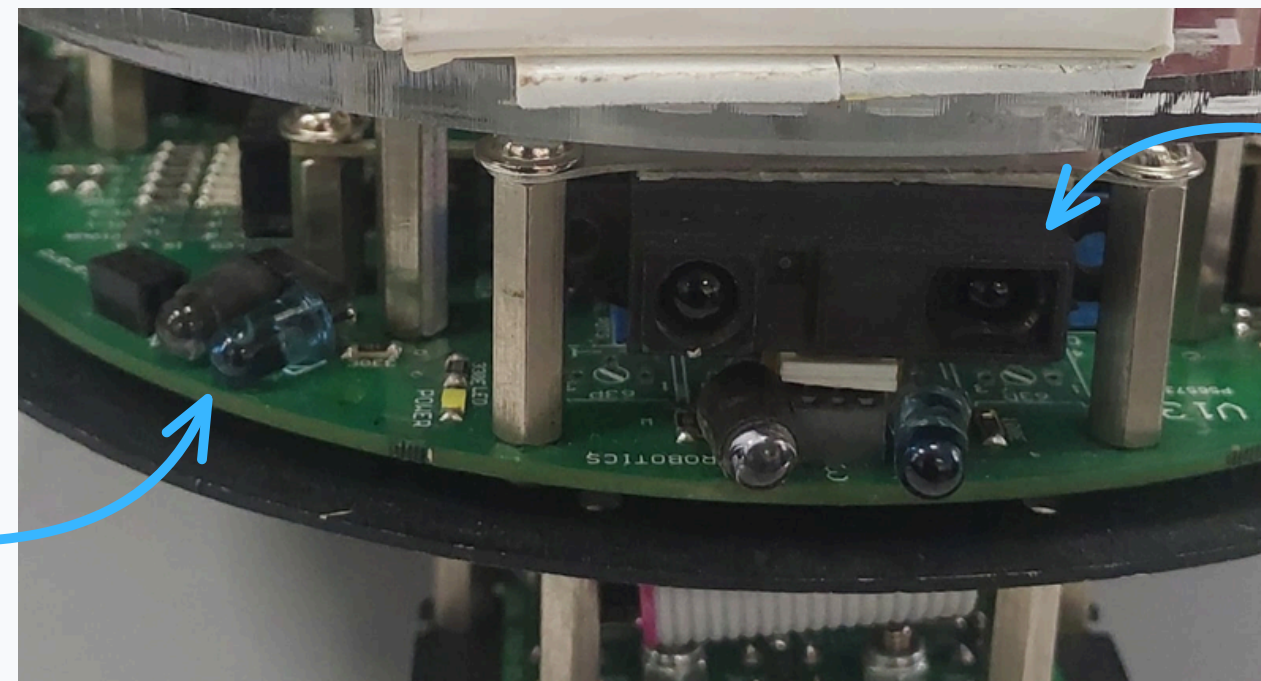
The Pi's camera doesn't see what's on the ground in front of the wheels. Rather than adding it to the vision pipeline, we gave the ATmega its own reflex.

Pi keeps streaming its usual command. If the front sensor sees something inside 15cm, the ATmega quietly overrides, runs the avoidance sequence, and resumes. The Pi never knows it happened.

Sharp GP2D12 (front) — analog IR, 15cm threshold, ADC converted via inverse power-law curve

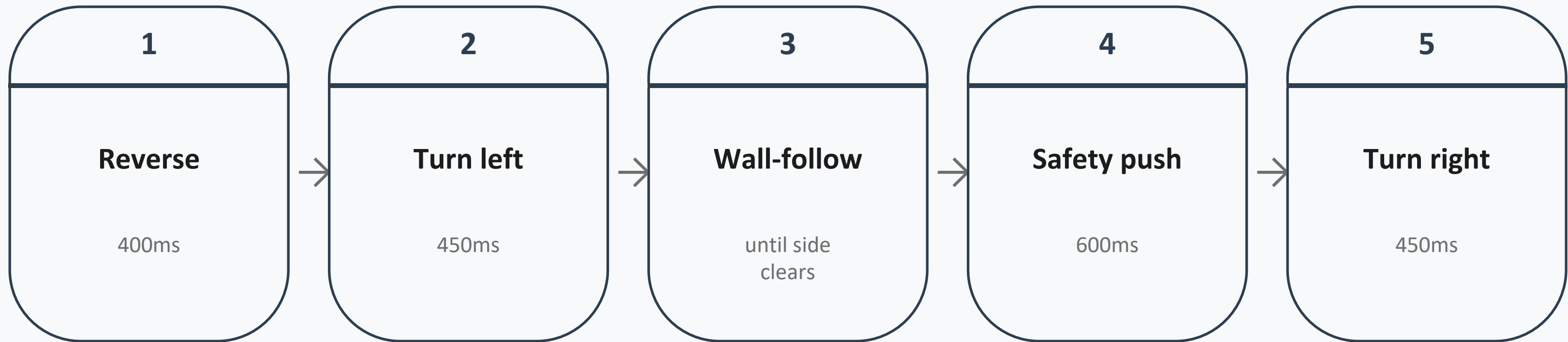
IR proximity (right side) — detects when the robot has cleared the obstacle during the manoeuvre

IR proximity



Sharp GP2D12

The avoidance sequence



Once complete, returns to idle → obeys Pi again → triggers again on next obstacle

Separate: tracking turns are 'burst' — 300ms turn, then 150ms pause to re-check the frame. Prevents overshoot when the target is already centred.

What actually changed

Metric	Phase 1 (MATLAB)	Phase 2 (Python)
Camera lag	8–10 s	~0 s (threaded)
Detection latency	700–800ms / frame	Model still ~700ms — but non-blocking
Detection target	COCO 'person'	Person + face-specific checkpoint
Steering	Binary L / R	3-band proportional (l / L / F / r / R)
Voice control	None	Offline start / stop (Vosk)
Obstacle avoidance	None	5-state ATmega FSM, 15cm threshold
Fail-safe	Watchdog (1s timeout)	Watchdog + voice STOP override

All Phase 2 test cases passed. Full observations in the report.

Future Plans/Improvements

Hardware

- 12V LiPo + buck converter (untethered)
- VL53L1X ToF for more reliable distance
- Pi Camera Module 3
- Second side sensor — bidirectional avoidance

Software

- ONNX / TFLite INT8 — push inference under 200ms
- Adaptive avoidance timings (not fixed durations)
- Expand voice vocabulary to directional commands
- Face recognition — follow a specific person
- Outdoor testing

Wrapping up

- Phase 1 delivered a working MATLAB/Simulink tracker — 1s latency, IoU locking, hardware watchdog.
-

- We tested 8 approaches and traced every failure to one root cause: Simulink's synchronous architecture.
-

- Phase 2 is finished — threaded camera, YOLOv8n, proportional steering, offline voice control, and ATmega obstacle avoidance, all running together.
-

- The system is layered: high-level perception on the Pi, low-level safety reflex on the ATmega underneath it.

Thanks — happy to take questions.